Archive de session du 16 octobre 2026 — Partie A : V17 et turbo par session ; Partie B : validation du turbo, benchmark et publication ; Partie C : monitoring Zêta

Auteur : hprzeta — résumé narratif de la session avec Claude (modèle Sonnet). Sans code : narration, données, conclusions et instructions de hprzeta. Capture brute du terminal : ***active*** *(processus* `script` *détecté), fichier* `claude-traitement-journalier/session_20261016_0915.log` *; elle se termine par* `exit` *deux fois.*

Partie A — première session de la journée

1. Contexte et règles de collaboration

Projet Riemann_Lab, branche de travail Riemann_Lab_IA. Règles rappelées : jamais de push forcé ; aucun `git add` ni commit sans validation explicite ; masquer tout secret ; copie de sauvegarde avant de modifier un script ; audit avant chaque push ; ne jamais lancer les calculs directement mais passer par `zeta_run.sh`. La session est la suite de celle du 15 octobre (V17 phase P0).

2. Première partie — `compute_zeros_v17.py` (étapes v17-a à d)

Contenu. Copie durcie de v16 : journal d'échecs structuré, validation LMFDB/Turing seulement si t_min ≤ 14, CSV de worker enrichis (provenance de chaque zéro), checkpoints périodiques atomiques avec fichier de progression, et fonction `consolider()` (variante A, sans STEP, recouvrement 0,5) à la place de la déduplication à 0,01. La revue a conduit à quatre corrections : erreur de consolidation sans repli silencieux, dossiers `consolidation_v17`, `_2`, `_3`, écriture finale obligatoire, échec d'un checkpoint périodique non fatal. Un contrôle d'alignement a été ajouté pour le risque D6 (non confirmé). Le défaut D4 (la quarantaine n'apparaît ni dans le CSV principal ni dans le journal) est **documenté seulement**, non corrigé.

Tests et mesures.

Mesure	Valeur
Tests de `consolider()`	16 réussis
Tests de `compute_zeros_v17.py`	9 réussis
Zéros bruts, rejeu T1b	185 821
Après consolidation	**185 814** (7 doublons confirmés, 4 paires ambiguës)
Comparaison avec le rejeu P0	identique ligne à ligne
Checkpoints bruts v16 contre v17	identiques bit à bit

Durée	33,9 min (turbo non actif : gouverneur resté en économie d'énergie)

Publication. Commit `b4526bf` (2 fichiers) poussé sur Riemann_Lab_IA, puis synchronisation autorisée des trois autres branches par fusions dans des arbres de travail temporaires : Riemann_Lab_C, main (`c35aaa9`) et Riemann_Lab_Test (`39f5ab2`). Tests relancés dans chaque branche, audit de secrets sans résultat.

3. Deuxième partie — turbo Zêta par session

Instruction de hprzeta (résumé verbatim).

> Le turbo doit être activé automatiquement au début de chaque nouvelle session Zêta, maintenu entre les runs, et désactivé uniquement au « bye bye » via le script de fin déjà présent ; restaurer l'état d'origine réel (pas un powersave imposé), idempotence, ne jamais prétendre que le turbo est actif sans vérification, ne pas contourner les autorisations système, tests sans modification réelle du gouverneur, revue obligatoire, et ne pas committer sans présenter le diff et les résultats de test.

Puis : « fais les 4 points que tu me recommandes ».

Constats d'audit (lecture seule).

- Le journal d'authentification ne montrait **aucun appel sudo turbo** depuis le démarrage du matin : le turbo n'avait jamais été exécuté, malgré des « turbo ok ».
- Anciens défauts : état écrasé à chaque activation, restauration imposant l'économie d'énergie, état dans un dossier temporaire, EPP non sauvegardé, coupure du turbo après chaque run.
- Faille existante : sudo sans mot de passe sur deux scripts situés dans un dossier modifiable par l'utilisateur et exécutés en root.

Les quatre points réalisés.

#	Recommandation	Réalisation
1	Déclencheur de session	crochet `SessionStart` local, affiche une ligne « Turbo Zêta : … »
2	Fin de session	étape 0 du « bye bye » : restauration vérifiée avant l'archive
3	Services	arrêtés seulement pendant les gros runs ; CPU et swappiness pendant la session
4	Durcissement sudoers	procédure fournie, **action root à faire par hprzeta**

Scripts. Activation et restauration réécrites (idempotentes, état initial enregistré une seule fois, restauration exacte vérifiée, secours explicite), un script de statut en lecture seule, une enveloppe de session sans privilège, et `zeta_run.sh` adapté. Tests : 34 réussis sur un système factice, 9 mutations toutes détectées (deux défauts trouvés en chemin : un cas de gouverneur intermédiaire non couvert, et un nom de service hostile pouvant être pris pour une option).

Publication. Commit turbo `1a9ad0c` poussé sur les quatre branches : Riemann_Lab_IA et Riemann_Lab_C `1a9ad0c`, main `5f3f5c4`, Riemann_Lab_Test `9e70cce`. Tests turbo relancés dans chaque branche fusionnée, local et origine identiques.

4. Documentation

Handoff local (bloc de reprise), JOURNAL et STACK du wiki mis à jour. Le wiki a été commité et poussé après validation : commit `2641cf9` (JOURNAL et STACK uniquement, audit sans résultat).

5. Test réel du turbo

Sur instruction « 2 » de hprzeta :

Contrôle	Résultat
Avant	inactif, 4 CPU en économie d'énergie
Activation	« ACTIF (vérifié : performance 4/4 CPU, swappiness 10) »
Deuxième activation	état initial conservé (idempotence confirmée)
Dérive après 20 s	aucune
Restauration au « bye bye »	**« RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial, swappiness 10) »**, code 0
État final	inactif, aucun état sauvegardé

6. Risques résiduels et suite

- Le durcissement sudoers n'est **pas fait** : la faille d'élévation de privilèges subsiste (action root de hprzeta).
- Pas de mesure du gain de performance ; dérive tardive possible par le gestionnaire de profils ; une seule session Zêta à la fois ; pas de restauration automatique après un plantage.
- Le crochet de session sera effectif à la prochaine ouverture.
- V17 : corriger D4, choisir le prochain run ; tests non faits (coupure d'alimentation réelle, disque plein, second poste, rescan avec vrai déficit).
- Ouverts, inchangés : rotation du jeton DuckDNS, nettoyage de `~/zeta_test_restore`, clone Toshiba (essai à blanc de l'option 8).
- Non commités, volontairement : `.gitignore`, archives d'octobre, `resultats_v17/`, scripts de métriques, `Dossier-md.md` du wiki.

7. Capture brute

Active : `claude-traitement-journalier/session_20261016_0915.log`, à terminer par `exit` deux fois (le second génère le `.md`).

Partie B — suite de la journée (soir) : validation du turbo, benchmark, publication

Session ouverte avec le turbo déjà activé automatiquement par le crochet de session : c'est la première activation réelle observée. Modèle : Sonnet.

8. Point projet

Sur la demande « point projet », le bloc de reprise du Handoff local a été affiché tel quel, puis résumé en trois lignes : version V17 (`compute_zeros_v17.py`), dernier commit `1a9ad0c`, prochaine action = durcissement sudoers (action root), test réel du turbo, puis choix de la suite V17. Le crochet de session avait annoncé « Turbo Zêta : ACTIF » : le point « non vérifié » du Handoff (aucune activation réelle) s'en trouvait en partie levé.

9. Cycle du turbo vérifié en réel

Instructions de hprzeta : « lance le status du turbo », puis « lance le test de end », puis « relance start puis teste zeta-run sur un petit T ».

Étape	Résultat mesuré
Statut après activation par le crochet	ACTIF : gouverneur et EPP en performance sur 4 CPU sur 4, swappiness 10, boost matériel actif, aucun service arrêté
Restauration (fin de session simulée)	« RESTAURÉ », code 0 ; 4 CPU revenus à l'économie d'énergie avec EPP d'origine ; état sauvegardé supprimé ; statut INACTIF
Réactivation	ACTIF
Run de test T=1000 via le script de run	13,5 s, code 0, services redémarrés, turbo CPU conservé

Piège noté : le script de statut renvoie le code 1 quand le turbo est inactif (ce n'est pas une erreur). Le Handoff a été mis à jour après chaque étape.

10. Anomalie avahi et bluetooth

Le run de test affichait « TURBO : PARTIEL » : les services avahi-daemon et bluetooth restaient actifs après leur arrêt.

Cause (mesurée, lecture seule). Les deux services sont relancés dans la seconde qui suit l'arrêt : avahi par sa socket d'activation (et par un service d'impression qui le demande), bluetooth par l'activation D-Bus. Preuve : leurs processus ont démarré au lancement du run et non à sa fin.

Essais.

Essai	Résultat
Arrêter aussi la socket avant le service	arrêt de la socket annulé (« Job canceled »), bluetooth toujours actif : 2 anomalies
Arrêter socket et service en une seule commande	pire : 4 anomalies, un client relance activement avahi (non identifié)
Décision de hprzeta : option 2	**abandonner l'arrêt de ces deux services**

Essai	Résultat

Réalisation : retour à la copie de sauvegarde, puis retrait d'avahi et de bluetooth de la liste d'arrêt ; trois tests qui utilisaient bluetooth comme exemple adaptés ; 34 tests réussis ; run réel T=1000 sans aucune anomalie (« TURBO : ACTIF »). Dans la même séquence, une erreur de ma part (mauvais chemin du Handoff) a été corrigée immédiatement.

11. Benchmark : le turbo accélère-t-il le calcul ?

Instruction de hprzeta : « mesure le gain du turbo », puis « lance le benchmark à T=20000 ». Protocole : même calcul dans les deux modes, trois alternances sans turbo / avec turbo, sans arrêt de services, pause avant chaque mesure, machine au repos.

T	Sans turbo (moyenne, en s)	Avec turbo (moyenne, en s)	Écart
5 000	9,83 (étendue 0,19)	9,63 (étendue 0,37)	environ 2 %, dans le bruit
20 000	37,56 (écart-type 0,72)	36,97 (écart-type 0,67)	environ 1,6 %, dans le bruit

Conclusion (mesure, prudente). Aucun gain démontré : l'écart est inférieur à la dispersion entre répétitions. Explication probable, non démontrée : sur cette machine, l'économie d'énergie avec EPP « balance_performance » monte déjà à environ 3,5 GHz sous charge ; la fréquence relevée en fin de run est identique dans les deux modes. Limites : trois mesures par mode, pas de test statistique, runs courts (10 à 40 s), fréquence relevée une seule fois. Le throttling thermique vu le 15 octobre (3,0 GHz vers 75 °C) concernait un run de 33 minutes, non reproduit ici.

Décision de hprzeta : garder le turbo de session (coût nul, bénéfice non démontré), à réévaluer sur un run long avec suivi de la fréquence.

12. Publication du correctif

Sur instructions « commite le script et les tests », puis « pousse fdc9506 et porte-le aux autres branches » :

Branche	Commit	Méthode
Riemann_Lab_IA	`fdc9506`	poussé
Riemann_Lab_C	`fdc9506`	avance rapide
main	`2f489ae`	fusion de IA dans un arbre de travail temporaire
Riemann_Lab_Test	`7e859f7`	fusion de IA dans un arbre de travail temporaire

Audit de secrets, adresses IP et MAC sans résultat, aucun push forcé, copies de sauvegarde ignorées par git. La copie de travail de hprzeta est restée intacte.

13. Nettoyage du Handoff

Trois mises à jour du Handoff (hash, bloc de reprise, lignes « non vérifié » et « non commité ») à la demande de hprzeta. Correction notable : le journal et la pile technique du wiki étaient déjà commités (`2641cf9`) alors que le Handoff les disait non commités ; le test réel de `zeta_run` avec arrêt des services a été retiré des points non vérifiés.

14. Étape 0 du « bye bye »

« Turbo Zêta : RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial (cpu0=powersave), swappiness 10) », code 0.

15. Risques résiduels et suite

- Durcissement sudoers toujours **non fait** (action root) ; il faudra aussi recopier les scripts à jour vers le dossier système.
- Gain du turbo non démontré sur les runs très longs ; comportement du gestionnaire de profils pendant un gros run non testé ; restauration après plantage non testée.
- V17 : choisir entre corriger D4 et lancer le run T=5M ; tests non faits (coupure d'alimentation, disque plein, second poste, rescan avec vrai déficit).
- Ouverts, inchangés : rotation du jeton DuckDNS, nettoyage de `~/zeta_test_restore`, clone Toshiba (essai à blanc de l'option 8).
- Non commités, volontairement : `.gitignore`, archives d'octobre, `resultats_v17/`, scripts de métriques, `Dossier-md.md` du wiki.

Partie C — nuit : monitoring Zêta, menu `zeta-script`, synchronisation des branches

Session ouverte après un effacement du contexte ; hprzeta a collé le prompt de mise en place du nouveau monitoring. Modèle : Sonnet. Le turbo n'était pas actif au moment du « bye bye » (voir section 21).

16. Consigne de hprzeta

Un dossier `monitoring/` venait d'être ajouté : une mesure par seconde (température, fréquence, charge, mémoire) des cinq machines, et, à la fin de chaque run, un rapport automatique (trois graphiques commentés, Markdown et PDF). Consignes : vérifier l'installation, tester les cinq machines, activer le logger et le détecteur de runs, mettre à jour `zeta-script` et sa documentation, ne pas modifier `monitoring/scripts/`, ne rien commiter sans accord, s'arrêter avant tout commit pour montrer le diff.

17. Étape 0 (mesuré)

- L'installation était déjà faite : les fonctions `zeta-mon-*` sont chargées par le fichier de démarrage du shell.
- Le test des machines a répondu OK pour PC1 à PC5, sans problème de SSH multiplexé, PC4 sous OpenBSD compris. La mémoire de PC4 n'est pas mesurée (attendu).

- Le logger et le détecteur de runs ont été activés ; les mesures des cinq machines ont été relues. Aucun run réel n'a été lancé. Le dernier run listé est l'ancien run historique T1b.

18. Mise à jour de `zeta-script`

- **Constat :** le menu ne lisait que les lignes `alias` du fichier de démarrage ; les nouvelles commandes sont des fonctions bash, donc invisibles.
- **Correction :** le menu lit maintenant aussi le fichier de fonctions du monitoring, sans dupliquer de code. Dix commandes apparaissent (numéros 37 à 46). Les trois qui ouvrent une fenêtre graphique sont marquées d'un avertissement (ne pas les lancer pendant un run). La saisie d'arguments est facultative. Une copie de sauvegarde du script a été faite avant modification.
- **Test :** le lancement de la commande d'état via le menu (numéro 39) fonctionne, après confirmation.
- **Documentation :** section « Menu zeta-script » et procédure avant / pendant / après un run ajoutées au `README.md` du monitoring (85 lignes).
- **Doublons signalés :** quatre anciens scripts de métriques dans `scripts/` font doublon avec ceux de `monitoring/scripts/` ; non touchés, à trancher par hprzeta.

19. Git

- Audit avant commit : aucun résultat (secrets, adresses, e-mails). Commit `2de45af` (deux fichiers : le script et le README), poussé sur `Riemann_Lab_IA`.
- Sur la demande « sync les autres branches » : simulation de fusion sans conflit, puis `Riemann_Lab_C` en avance rapide, `main` (`9c44c2b`) et `Riemann_Lab_Test` (`e07feec`) par fusions avec commit de fusion, dans un répertoire de travail temporaire pour ne pas toucher aux fichiers non commités de hprzeta. Aucune option de forçage.
- Les branches `inbox-ia`, `session` et la branche d'exercice sur la formule d'Euler ne font pas partie des quatre branches synchronisées.

20. Suivi mis à jour et publié

Sur la demande « on fera le test demain, mets à jour Handoff, Stack, etc. » : bloc de reprise du Handoff privé, résumé du Handoff du wiki, nouveau pavé dans le journal, nouvelle section dans la pile technique, entrée dans la mémoire du projet. Puis, sur « go », audit (une seule occurrence : le mot « DuckDNS » dans « jeton DuckDNS », sans valeur secrète) et commit `2ff8060` du wiki, poussé sans forçage. Le Handoff privé reste hors dépôt.

21. Étape 0 du « bye bye » et capture

- Restauration du turbo : « Turbo Zêta : RIEN À RESTAURER ». L'état relu montre les processeurs en `powersave` avec la préférence d'énergie d'origine : le turbo de session n'avait donc pas été activé dans cette session (le crochet de démarrage n'a pas laissé d'état). Cause non investiguée.
- Capture brute : active (processus `script` détecté), fichier `claude-traitement-journalier/session_20261016_0915.log` (le même que pour les parties A et B) ; à terminer par `exit` deux fois.

22. Risques résiduels et suite

- **Non vérifié :** détection d'un vrai run par le détecteur et génération réelle du rapport ; présence de la bibliothèque PDF dans l'environnement Python ; correspondance entre le nom du script lancé par `zeta_run.sh` et le motif `compute_zeros`.
- **Demain (17/10) :** vérifier l'état du monitoring, nommer le run, lancer un run court pour valider la chaîne avant tout gros run, sans fenêtre graphique pendant le run.
- **À décider :** publication du reste de `monitoring/` (code et page wiki « Monitoring »), sort des quatre scripts en doublon.
- Inchangés : durcissement sudoers (action root), choix de la suite V17, rotation du jeton DuckDNS, nettoyage de `~/zeta_test_restore`, clone Toshiba (essai à blanc de l'option 8).
- Non commités, volontairement : `.gitignore`, archives d'octobre, `resultats_v17/`, `monitoring/` hors README, anciens scripts de métriques, `Dossier-md.md` du wiki.